

TOOLS, TECHNOLOGIES & CROSS-PLATFORM APP FEASIBILITY

Smart Rodent Bait Station

Prepared by Sadia Javed — BS Artificial Intelligence, Bahria University Islamabad

Table of Contents

Table of Contents	1
1. Tools & Technologies	2
1.1 Edge & firmware	2
1.2 AI / object detection	2
1.3 Backend / cloud	2
1.4 Mobile app (cross-platform)	3
1.5 Web dashboard	3
1.6 Development & collaboration tools	3
2. Cross-Platform Mobile App Feasibility	4
2.1 Is cross-platform feasible for this app?	4
2.2 Framework options considered	4
2.3 Recommended approach: Flutter	4
2.4 Feasibility by constraint	5
3. Summary	5

1. Tools & Technologies

The table below lists the complete tool/technology stack across all layers of the system — firmware, AI model, backend, and the two client applications.

1.1 Edge & firmware

Category	Tool / Technology	Purpose
Microcontroller	ESP32-S3	On-device AI inference, Wi-Fi/BLE connectivity, sensor I/O
Firmware framework	ESP-IDF / Arduino core	Low-level firmware development for ESP32-S3
Sensors	IR break-beam, IR camera, load cell, tamper switch	Entry detection, image capture, bait monitoring, tamper alerts
Connectivity	Wi-Fi (primary), 4G/LTE or LoRaWAN (fallback)	Transmit detection events to the cloud

1.2 AI / object detection

Category	Tool / Technology	Purpose
Model	YOLOv8-nano	Lightweight rat/mouse object detection
Runtime	TensorFlow Lite Micro	Quantized on-device inference on the ESP32-S3
Training	Python, Ultralytics YOLO, Roboflow	Dataset annotation, model training, export to TFLite

1.3 Backend / cloud

Category	Tool / Technology	Purpose
Backend platform	Firebase	Auth, real-time database, storage, serverless functions, push notifications
Database	Cloud Firestore	Real-time event and device metadata storage
Media storage	Firebase Cloud Storage	Stores captured images/clips per detection event
Serverless logic	Firebase Cloud Functions	Alert routing, report generation, validation
Push notifications	Firebase Cloud Messaging (FCM)	Real-time alerts to mobile devices

1.4 Mobile app (cross-platform)

Category	Tool / Technology	Purpose
Framework	Flutter (Dart)	Single codebase compiled to native Android and iOS apps
State management	Riverpod / Provider	Reactive UI bound to live Firestore data
Local storage	Hive / SQLite	Offline caching of last-known station states
Charts	fl_chart / syncfusion_flutter_charts	Activity graphs and analytics on mobile

1.5 Web dashboard

Category	Tool / Technology	Purpose
Framework	React + Tailwind CSS	Component-based admin dashboard UI
Charts	Recharts / Chart.js	Live KPI graphs and activity visualizations
Real-time sync	Firestore onSnapshot listeners	Live data updates without polling

1.6 Development & collaboration tools

Category	Tool / Technology	Purpose
Version control	Git + GitHub	Source control and team collaboration
IDE	VS Code, Android Studio	Flutter/firmware development and debugging
Design	Figma	UI/UX mockups for mobile app and dashboard
Project tracking	Trello / Notion	Sprint planning and task tracking

2. Cross-Platform Mobile App Feasibility

The project requires a single mobile app that runs natively on both Android and iOS. This section evaluates whether that is feasible, what approach to take, and why.

2.1 Is cross-platform feasible for this app?

Yes. The mobile app's requirements (station list/map, live status, push notifications, activity charts, basic settings) are standard data-driven UI patterns — none require deep platform-specific native code such as custom camera pipelines, AR, or low-level hardware drivers on the phone itself (all hardware-level work happens on the ESP32-S3, not the phone). This makes the app a strong candidate for cross-platform frameworks rather than building two separate native apps.

2.2 Framework options considered

Approach	Pros for this project	Cons for this project
Flutter	Single Dart codebase for Android + iOS; first-party Firebase SDKs; strong charting libraries; consistent pixel-perfect UI across both platforms; AOT compilation gives fast, predictable performance	Smaller native-module ecosystem than React Native for highly specialized hardware bridges (not a concern here)
React Native	Large ecosystem and community; good fit if the team already knows React/JavaScript	Requires a separate UI adaptation mindset for Android vs iOS; team has no existing React codebase to leverage
Native (Kotlin + Swift)	Maximum platform-specific control and performance	Two full codebases to build, test, and maintain — roughly double the development time for a Week-1-to-graduation timeline

2.3 Recommended approach: Flutter

- One codebase, two app stores: Flutter compiles to native ARM binaries for both Android and iOS from a single Dart codebase, which directly satisfies the cross-platform requirement without maintaining parallel codebases.
- Performance: Flutter's Impeller rendering engine draws every pixel itself rather than relying on platform UI bridges, giving consistent 60+ fps and sub-300ms cold-start times on modern devices — well within what a real-time monitoring app needs.
- Firebase fit: since the recommended backend is Firebase, Flutter's first-party Firebase plugins (firebase_auth, cloud_firestore, firebase_messaging, firebase_storage) integrate with minimal glue code.
- UI consistency: the dashboard-style screens in this app (KPI cards, maps, charts, lists) are exactly the kind of UI Flutter is strongest at rendering identically on both platforms.
- Team fit: no existing React/JavaScript codebase to leverage, so React Native's main advantage (team reuse) does not apply here — removing that as a deciding factor in its favor.

Verdict: Flutter is the most feasible choice for this project's cross-platform mobile app — it satisfies the Android + iOS requirement from one codebase, integrates cleanly with the recommended Firebase backend, and matches the dashboard-heavy UI this app needs.

2.4 Feasibility by constraint

Constraint	Assessment
Technical	Feasible. All required features (live data, push notifications, maps, charts, offline caching) are well-supported by mature Flutter packages.
Time	Feasible within an academic-project timeline. One codebase for both platforms roughly halves UI development time compared to building separate native apps.
Skill / learning curve	Manageable. Dart syntax is close to other OOP languages already covered in the BS-AI curriculum (Java-like structure); abundant official documentation and codelabs exist.
Cost	Feasible. Flutter, Firebase's free tier (Spark plan), and all listed tools are free for development and small-scale deployment.
Deployment	Feasible, with one caveat: publishing to the Apple App Store requires a Mac (or a cloud Mac service such as Codemagic/MacStadium) for the iOS build and an Apple Developer account.

Note on iOS builds: Flutter can be developed entirely on Windows/Linux for Android, but producing the final iOS build/IPA still requires Xcode on macOS at some point in the pipeline. If no Mac is available, a cloud build service (e.g. Codemagic's free tier) can produce the iOS build without owning physical Apple hardware.

3. Summary

- A single cross-platform mobile app for Android and iOS is technically and practically feasible for this project.
- Flutter is the recommended framework, primarily due to single-codebase efficiency, strong Firebase integration, and fit with the dashboard-style UI this app requires.
- The only feasibility caveat is the need for a macOS build step (local or cloud-based) to publish to the Apple App Store — this does not block development, only final iOS deployment.